

Matrix Multiplication Example

- Major Cache Effects to Consider
 - Total cache size
 - Exploit temporal locality and keep the working set small (e.g., by using blocking)
 - Block size
 - Exploit spatial locality
- Description:
 - Multiply $N \times N$ matrices
 - $O(N^3)$ total operations
 - Accesses
 - N reads per source element
 - N values summed per destination
 - but may be able to hold in register

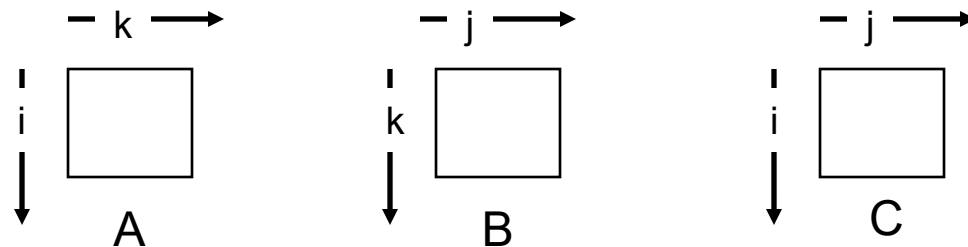
```
/* ijk */
for (i=0; i<n; i++) {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

*Variable sum
held in register*



Miss Rate Analysis for Matrix Multiply

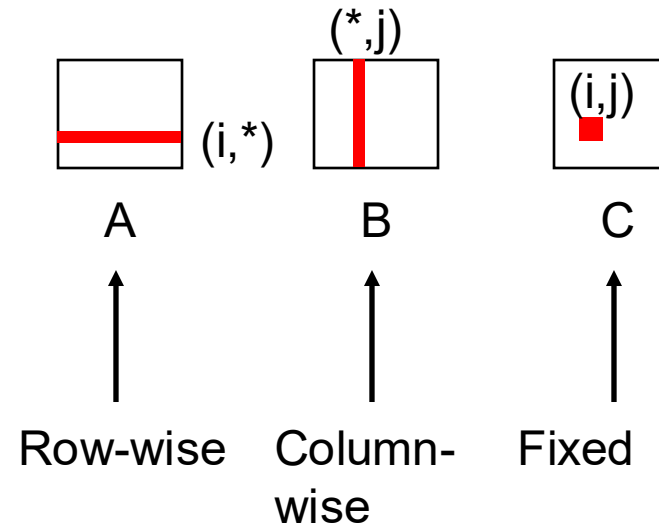
- Assume:
 - Line size = 32B (big enough for 4 64-bit words)
 - Matrix dimension (N) is very large
 - Approximate $1/N$ as 0.0
 - Cache is not even big enough to hold multiple rows
- Analysis Method:
 - Look at access pattern of inner loop



Matrix Multiplication (ijk)

```
/* ijk */
for (i=0; i<n; i++) {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

Inner loop:



Misses per Inner Loop Iteration:

<u>A</u>	<u>B</u>	<u>C</u>
0.25	1.0	0.0

Matrix Multiplication (jik)

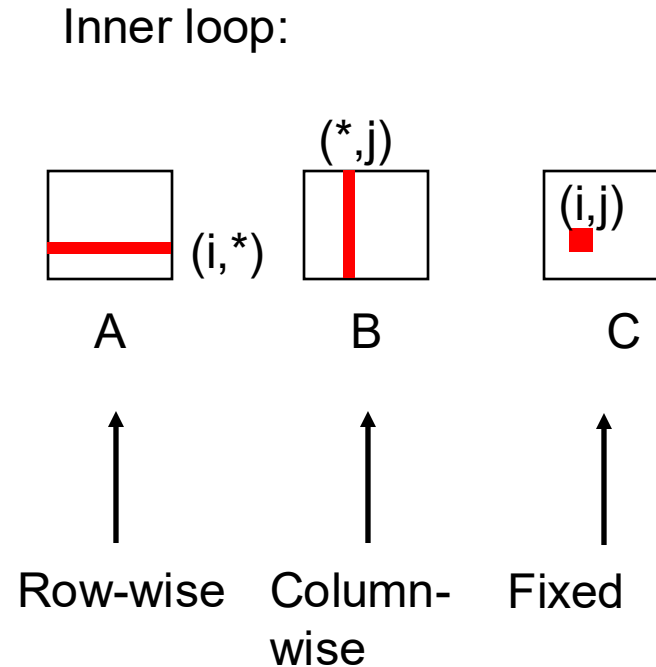
```
/* jik */
for (j=0; j<n; j++) {
  for (i=0; i<n; i++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum
  }
}
```

Misses per Inner Loop Iteration:

A
0.25

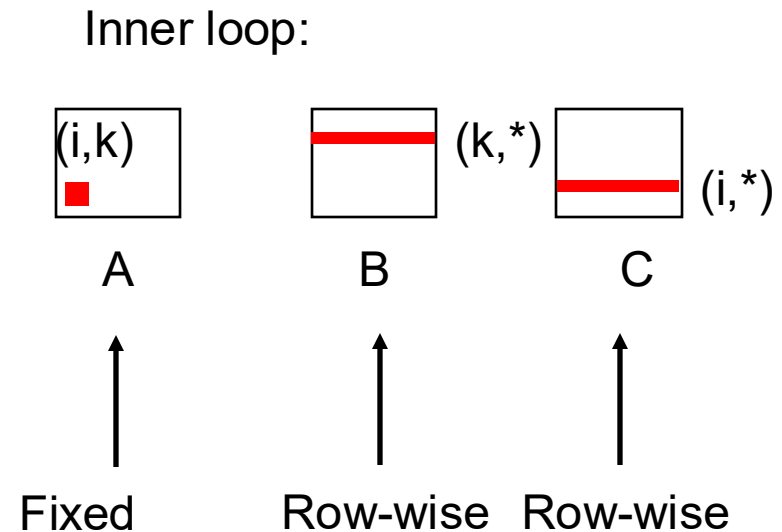
B
1.0

C
0.0



Matrix Multiplication (kij)

```
/* kij */
for (k=0; k<n; k++) {
  for (i=0; i<n; i++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```



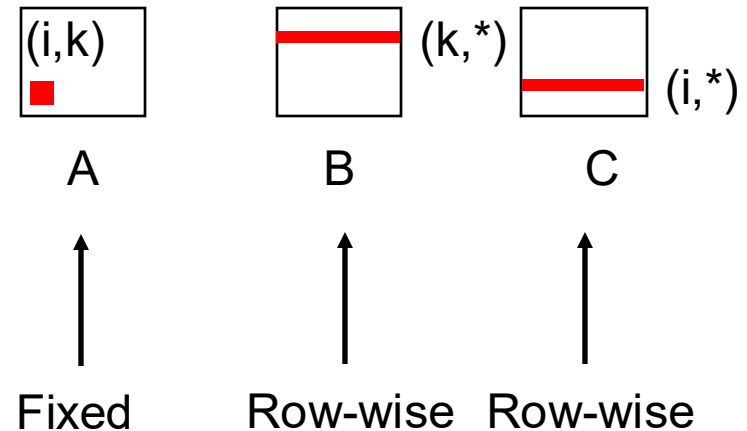
Misses per Inner Loop Iteration:

<u>A</u>	<u>B</u>	<u>C</u>
0.0	0.25	0.25

Matrix Multiplication (ikj)

```
/* ikj */  
for (i=0; i<n; i++) {  
  for (k=0; k<n; k++) {  
    r = a[i][k];  
    for (j=0; j<n; j++)  
      c[i][j] += r * b[k][j];  
  }  
}
```

Inner loop:



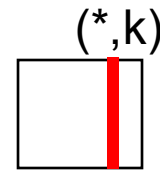
Misses per Inner Loop Iteration:

<u>A</u>	<u>B</u>	<u>C</u>
0.0	0.25	0.25

Matrix Multiplication (jki)

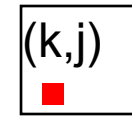
```
/* jki */  
for (j=0; j<n; j++) {  
  for (k=0; k<n; k++) {  
    r = b[k][j];  
    for (i=0; i<n; i++)  
      c[i][j] += a[i][k] * r;  
  }  
}
```

Inner loop:



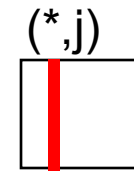
A

Column -
wise



B

Fixed



C

Column-
wise

Misses per Inner Loop Iteration:

A

1.0

B

0.0

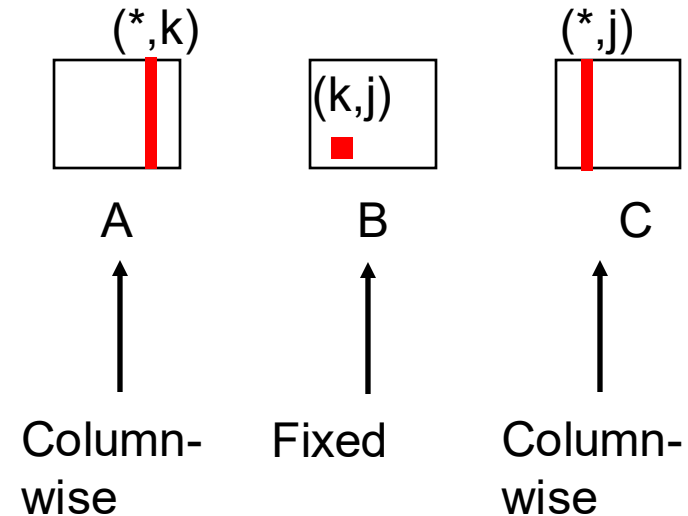
C

1.0

Matrix Multiplication (kji)

```
/* kji */  
for (k=0; k<n; k++) {  
  for (j=0; j<n; j++) {  
    r = b[k][j];  
    for (i=0; i<n; i++)  
      c[i][j] += a[i][k] * r;  
  }  
}
```

Inner loop:



Misses per Inner Loop Iteration:

A
1.0

B
0.0

C
1.0

Summary of Matrix Multiplication

ijk (& jik):

- 2 loads, 0 stores
- misses/iter = 1.25

kij (& ikj):

- 2 loads, 1 store
- misses/iter = 0.5

jki (& kji):

- 2 loads, 1 store
- misses/iter = 2.0

```
for (i=0; i<n; i++) {  
    for (j=0; j<n; j++) {  
        sum = 0.0;  
        for (k=0; k<n; k++)  
            sum += a[i][k] * b[k][j];  
        c[i][j] = sum;  
    }  
}
```

```
for (k=0; k<n; k++) {  
    for (i=0; i<n; i++) {  
        r = a[i][k];  
        for (j=0; j<n; j++)  
            c[i][j] += r * b[k][j];  
    }  
}
```

```
for (j=0; j<n; j++) {  
    for (k=0; k<n; k++) {  
        r = b[k][j];  
        for (i=0; i<n; i++)  
            c[i][j] += a[i][k] * r;  
    }  
}
```

Improving Temporal Locality by Blocking

- Example: Blocked matrix multiplication
 - “block” (in this context) does not mean “cache block”.
 - Instead, it means a sub-block within the matrix.
 - Example: $N = 8$; sub-block size = 4

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

Key idea: Sub-blocks (i.e., A_{xy}) can be treated just like scalars.

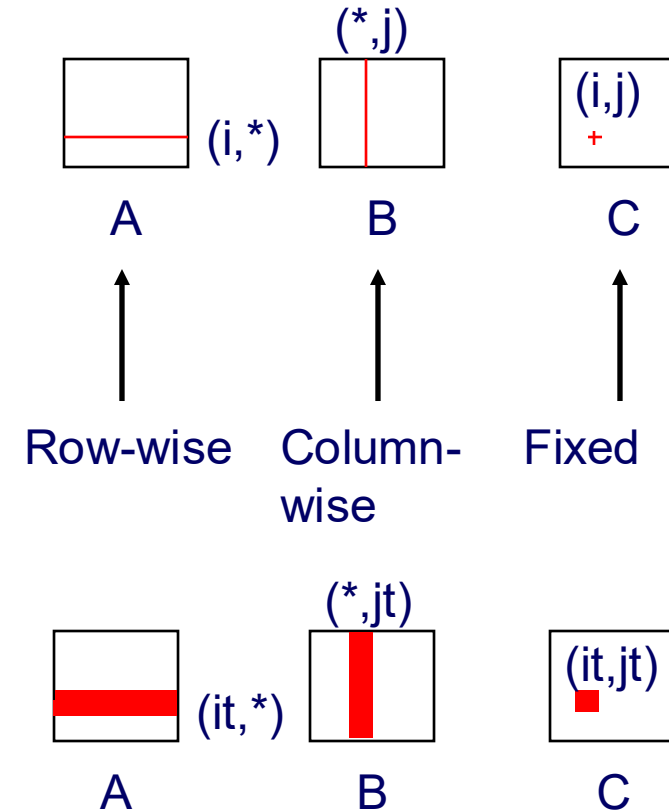
$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} & C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} & C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

Blocked Matrix Multiplication

```
/* ijk */
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        for (k=0; k<n; k++)
            c[i][j] += a[i][k]*b[k][j];
```

```
/* Tiled; assume n multiple of T */
for (it=0; it<n; it+=T)
    for (jt=0; jt<n; jt+=T)
        for (kt=0; kt<n; kt+=T)
            for (i=it; i<it+T; i++)
                for (j=jt; j<jt+T; j++)
                    for (k=kt; k<kt+T; k++)
                        c[i][j] += a[i][k]*b[k][j];
```

Inner loop:



Cache Misses: Blocked Mat-Mult

```
/* Tiled; assume n multiple of T */  
for (it=0; it<n; it+=T)  
  for (jt=0; jt<n; jt+=T)  
    for (kt=0; kt<n; kt+=T)  
      for (i=it; i<it+T; i++)  
        for (j=jt; j<jt+T; j++)  
          for (k=kt; k<kt+T; k++)  
            c[i][j] += a[i][k]*b[k][j];
```

Assume fully associative
Cache of size $> 3 \cdot T \cdot T$

sub-mat-mult



Tiling = Loop-Split+Permutation

```
/* ijk */  
for (i=0; i<n; i++)  
  for (j=0; j<n; j++)  
    for (k=0; k<n; k++)  
      c[i][j] += a[i][k]*b[k][j];
```

Strip-mine each loop into
a pair of equivalent loops



```
for (it=0; it<n; it+=T)  
  for (i=it; i<it+T; i++)  
    for (jt=0; jt<n; jt+=T)  
      for (j=jt; j<jt+T; j++)  
        for (kt=0; kt<n; kt+=T)  
          for (k=kt; k<kt+T; k++)  
            c[i][j] += a[i][k]*b[k][j];
```

```
for (it=0; it<n; it+=T)  
  for (jt=0; jt<n; jt+=T)  
    for (kt=0; kt<n; kt+=T)  
      for (i=it; i<it+T; i++)  
        for (j=jt; j<jt+T; j++)  
          for (k=kt; k<kt+T; k++)  
            c[i][j] += a[i][k]*b[k][j];
```

Loop Permutation



Impact of Memory Usage on Occupancy

- The memory usage of a kernel plays an important role in occupancy tuning.
- While CUDA registers and shared memory can be extremely effective at reducing the number of accesses to global memory, one must be careful to stay within the SM's capacity of these memories.
- Each CUDA device offers limited resources, which limits the number threads that can simultaneously reside in the SM for a given application.
- In general, the more resources each thread requires, the fewer the number of threads that can reside in each SM.

How Shared memory usage can also limit the number of threads that can be assigned to each SM?

- For example, the A100 GPU can be configured to have up to 164 KB of shared memory per SM and supports a maximum of 2048 threads per SM.
- Thus for all 2048 thread slots to be used, a thread block should not use more than an average of $(164 \text{ KB}) / (2048 \text{ threads}) = 82 \text{ B/thread}$.
- In the tiled matrix multiplication example, every block has TILE_WIDTH^2 threads, and uses $\text{TILE_WIDTH}^2 * 4\text{B}$ of shared memory for Mds and $\text{TILE_WIDTH}^2 * 4\text{B}$ of shared memory for Nds.
- Thus the thread block uses an average of $(\text{TILE_WIDTH}^2 * 4\text{B} + \text{TILE_WIDTH}^2 * 4\text{B}) / (\text{TILE_WIDTH}^2 \text{ threads}) = 8 \text{ B/thread}$ of shared memory.
- Therefore the tiled matrix multiplication kernel's occupancy is not limited by the shared memory.

How Shared memory usage can also limit the number of threads that can be assigned to each SM? (Contd.)

- However, consider a kernel that has thread blocks that use 32 KB of shared memory, each of which has 256 threads.
- In this case, the kernel uses an average of $(32 \text{ KB}) / (256 \text{ threads}) = 132 \text{ B/thread}$ of shared memory.
- With such shared memory usage, the kernel cannot achieve full occupancy. Each SM can host a maximum of only $(164 \text{ KB}) / (132 \text{ B/thread}) = 1272 \text{ threads}$.
- Therefore the maximum achievable occupancy of this kernel will be $(1272 \text{ assigned threads}) / (2048 \text{ maximum threads}) = 62\%$.

Dynamic Allocation of Shared Memory

- It is often desirable for a kernel to be able to use different amounts of shared memory according to the amount available in the hardware.
 - That is, we may want a host code to dynamically determine the size of the shared memory and adjust the amount of shared memory that is used by a kernel.
- This can be done by calling the `cudaGetDeviceProperties` function. Assume that variable `&devProp` is passed to the function.
- In this case, the field `devProp.sharedMemPerBlock` gives the amount of shared memory that is available in each SM.
- The programmer can then determine the amount of shared memory that should be used by each block.

Dynamic Allocation of Shared Memory (Contd.)

- We can enable such adjustment with a different style of declaration in CUDA by adding a `Cextern` keyword in front of the shared memory declaration and omitting the size of the array in the declaration.
- Based on this style, the declarations for Mds and Nds need to be merged into one dynamically allocated array:

```
extern __shared__ Mds_Nds[];
```

Dynamic Allocation of Shared Memory (Contd.)

```
size_t size =  
calculate_appropriate_SM_usage(devProp.sharedMemPerBlock,  
...);  
  
matrixMulKernel<<<dimGrid,dimBlock,size>>>(Md, Nd, Pd,  
Width, size/2, size/2);
```

```
01  #define TILE_WIDTH 16  
02  __global__ void matrixMulKernel(float* M, float* N, float* P, int Width,  
                                unsigned Mdz_sz, unsigned Nds_sz) {  
03  
04  extern __shared__ char float Mds_Nds[];  
05  
06  float *Mds = (float *) Mds_Nds;  
07  float *Nds = (float *) Mds_Nds + Mds_sz;
```